BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN ROMANIAN

DIPLOMA THESIS

ASR for Medicine: Optimizing Speech Recognition for Medical Conversations

Supervisor
Prof. Czibula Istvan Gergely

Author
Amarie Tiberiu

2025

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ÎN LIMBA ROMÂNĂ

LUCRARE DE LICENŢĂ

ASR pentru Medicină: Optimizarea Recunoașterii Vocale pentru Conversații Medicale

Coordonator științific
Prof. Czibula Istvan Gergely

Absolvent
Amarie Tiberiu

2025

ABSTRACT

The administrative burden of clinical documentation is a significant contributor to physician burnout and inefficiency in healthcare. This thesis addresses this challenge by first reviewing the landscape of Automatic Speech Recognition (ASR) technologies and then proposing a solution to optimize a state-of-the-art model for medical conversations. The core of this work focuses on enhancing the general-purpose Whisper model for the specialized domain of medical dialogue.

To achieve this, we investigate and apply Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning strategy, which allows for effective model adaptation with limited computational resources. Using a synthetic medical dataset due to the privacy constraints of real-world data, our fine-tuned model achieved a Word Error Rate (WER) of 8.26%, a notable improvement over the 8.75% WER of the original base model.

To showcase the practical application of this research, the enhanced model was first deployed as a robust RESTful API for external use. Subsequently, this API was integrated into `medspeech.io`, a purpose-built web application designed for clinicians. The platform enables physicians to record, transcribe, and manage patient conversations, serving as a comprehensive proof-of-concept for reducing administrative workloads. This thesis successfully validates that targeted fine-tuning can significantly improve ASR performance in specialized domains and delivers a functional prototype poised for future clinical integration and evaluation.

Contents

1 Introduction **1**
- 1.1 Context and motivation . . . 1
- 1.2 Objectives . . . 2
- 1.3 Thesis structure . . . 2
- 1.4 Usage of Generative AI Tools . . . 3

2 Automatic Speech Recognition **4**
- 2.1 Sound processing . . . 5
- 2.2 Speech Recognition Methods . . . 9
  - 2.2.1 Hidden Markov Models (HMMs) . . . 9
  - 2.2.2 End-to-end (E2E) Deep Learning Based Methods . . . 13
- 2.3 Evaluation Metrics . . . 18

3 Implementation of a Medical Domain ASR System **20**
- 3.1 Fine-Tuning an ASR model on medical data . . . 20
  - 3.1.1 Problem Definition . . . 21
  - 3.1.2 Choosing a base ASR model . . . 21
  - 3.1.3 Data aquisition and preparation . . . 25
  - 3.1.4 Fine-Tuning Strategy: Low Rank Adaptation (LoRA) . . . 29
  - 3.1.5 Experimental Setup and Hyperparameters . . . 31
  - 3.1.6 Results . . . 32
  - 3.1.7 Model Deployment for inference . . . 35
- 3.2 Web Application . . . 36
  - 3.2.1 Requirements Specification . . . 37
  - 3.2.2 Implementation . . . 40
  - 3.2.3 App Flows . . . 42

4 Conclusions **44**
- 4.1 Future work . . . 45

Bibliography **46**

Chapter 1

Introduction

1.1 Context and motivation

The intersection of healthcare and artificial intelligence has emerged as a critical frontier in modern medical practice, with the potential to address longstanding challenges in clinical workflow efficiency, documentation quality, and ultimately patient care. Automatic speech recognition (ASR) technology represents one of the most promising applications in this domain, offering a solution to the burdensome documentation requirements that have become a significant source of physician burnout and inefficiency in healthcare delivery systems worldwide.

Medical professionals dedicate a substantial portion of their working hours —estimated between 34% and 55% according to recent studies—to clinical documentation tasks rather than direct patient care[SCL+16]. This administrative burden not only reduces the time available for meaningful patient interaction but also contributes significantly to professional dissatisfaction and burnout physicians[SDS+16]. Traditional solutions, including manual transcription services and early-generation dictation systems, have proven inadequate in addressing these challenges due to their cost, latency, and limited accuracy when confronted with specialized medical terminology.

The application of advanced speech recognition technologies in clinical settings presents a compelling opportunity to transform the documentation process. By automating the transcription of doctor-patient conversations, ASR systems can potentially reduce the administrative workload of healthcare professionals, allowing them to redirect their attention and expertise toward patient care. Furthermore, accurate transcription can enhance the completeness and quality of medical records, which serve as the foundation for clinical decision-making, medical education, and research.

1.2 Objectives

The primary objectives of this thesis are systematically structured to bridge the gap between theoretical research and practical application in the field of medical speech recognition. The specific goals are as follows:

- To conduct an extensive literature review of established and modern speech recognition methods, from traditional statistical models like HMMs to contemporary deep learning architectures.
- To investigate and apply the Low-Rank Adaptation (LoRA) strategy as a parameter-efficient method for fine-tuning large-scale transformer models, specifically targeting the Whisper architecture.
- To enhance the performance of the state-of-the-art Whisper model for medical speech recognition by fine-tuning it on a specialized dataset of clinical conversations, aiming for a measurable improvement in transcription accuracy.
- To develop and deploy a full-featured web application, `medspeech.io`, that integrates the fine-tuned model. This application will provide physicians with tools for recording, transcribing, and managing patient conversations, thereby demonstrating the practical value and improved capabilities of the specialized model.

1.3 Thesis structure

This thesis is composed of four chapters covering both theoretical foundations and practical implementation of speech recognition in medical applications. Chapter 1 introduces the topic and motivation.

Chapter 2 conducts extensive research over the speech recognition domain, covering sound processing and various recognition approaches from Hidden Markov Models and their variations to deep learning models with Connectionist Temporal Classification and Sequence-to-Sequence architectures.

Chapter 3 focuses on the practical aspects of this thesis. In the first part of this chapter, we implement the entire lifecycle of a machine learning project from problem definition to model deployment, including the Whisper architecture overview, data gathering and preparation, improving the capabilities of the Whisper base model using LoRA parameter-efficient fine-tuning strategy, results analysis, and model deployment. Section 3.1.7 details the integration of the enhanced model into a RESTful API for external use. The second part of this chapter presents the development of *medspeech.io*, a web application that integrates the improved Whisper model to enable

physicians to manage their patients, record and transcribe patient conversations, and maintain comprehensive medical notes.

Chapter 4 provides the conclusions and evaluation of the research and implementation results.

1.4 Usage of Generative AI Tools

During the preparation of this research, the author used ChatGPT and Google Gemini to reformulate certain paragraphs of the written paper and generate generic code in the practical application. After using this tool, the author reviewed and edited the generated content and assumes full responsibility for the content of the research.

Chapter 2

Automatic Speech Recognition

Speech recognition, also known as Automatic Speech Recognition (ASR), is the process that enables computers to receive, interpret, and transform human speech into written text. It is an interdisciplinary subfield located at the intersection between computational linguistics, signal processing, computer science and artificial intelligence. In human-computer interaction, automatic speech recognition aims to remove communication barriers by allowing spoken language to serve as a direct input channel. Today, speech recognition technology is integrated in a vast array of applications such as voice-controlled smart home and automotive devices, healthcare documentation, accessibility tools for individuals with disabilities, virtual assistants, making human-computer interaction more natural and accessible.

Research into automatic speech recognition started over 70 years ago and remains a relevant and active field of study today. The development began in the early 1950s with Bell Labs' "Audrey," a system capable of recognizing digits spoken by a single voice. This was followed in the 1960s by IBM's "Shoebox," which expanded the recognized vocabulary to 16 words. The field advanced significantly in the 1970s with the development of hidden Markov models and statistical approaches, enabling speech translation with vocabularies of over 1,000 words. The 2000s brought a paradigm shift in this domain with machine learning techniques and neural network usage that could model complex non-linear relationships between acoustic features and phonemes. Another leap was marked in the 2010s, with deep learning and recurrent neural networks dramatically improving accuracy, culminating in systems like Apple's Siri, and Amazon's Alexa that could handle natural conversational speech. Today, modern speech recognition, driven by transformer architectures and large datasets, has reached near-human levels of accuracy. However, challenges remain in areas such as low-resource languages, poor audio quality, and specialized domains like medicine and law.

2.1 Sound processing

Sound waves are essentially variations in air pressure that propagate through space. By nature, sound waves are continuous signals, which means that they contain an infinite number of values in a given time. These continuous signals, often referred to as **analog** signals, are what the human ear perceives as sound.

To enable digital devices to process these continuous sound waves, several steps must be taken to convert the analog signal to a digital representation, also known as a **digital** signal. Essentially, we need to convert the continuous signal into a finite array of discrete values that computers can process. This conversion must be performed as accurately as possible to create a discrete representation of the original sound waves without introducing excessive noise.

The process begins with a microphone that captures the analog sound waves and converts them into continuous electrical voltage signals. These electrical signals are then processed by an Analog-to-Digital Converter (ADC). The ADC uses two key procedures, **sampling** and **quantization**, to transform the continuous electrical signal into its final digital format.

Sampling

Sampling is the technique of converting a continuous signal, denoted as $S(t)$, into a sequence of discrete values. This is achieved by measuring the signal's instantaneous value at consistent time intervals. The duration between these measurements is called the **sampling period**, T. This process results in a series of samples, mathematically represented as $s(nT)$, where n is the sample index.

The **sampling rate**, f_s, is the inverse of the sampling period ($f_s = 1/T$) and indicates the number of samples captured per second, measured in Hertz (Hz).

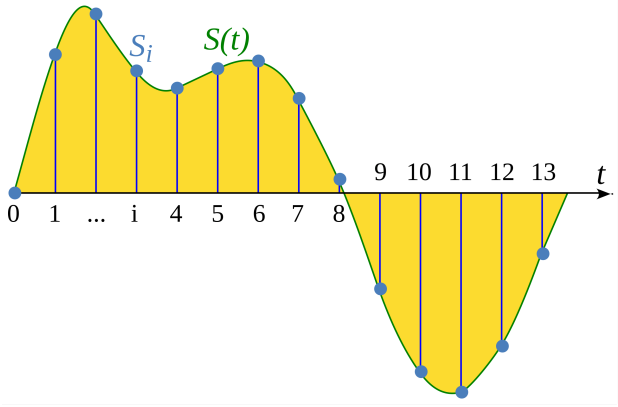


Figure 2.1: Sampling (signal processing)[Wik25]

The highest frequency that can be accurately captured is determined by the **Nyquist**

limit, which is half of the sampling rate. For human speech, all critical frequencies are under 8 kHz, making a 16 kHz sampling rate ideal. Using a higher rate offers no audible improvement for speech and increases computational load, while a lower rate, like 8 kHz, would result in muffled, poor-quality audio due to the loss of high-frequency information.

Quantization

In digital audio, each audio sample records the **amplitude** of the audio wave at a point in time. Sound itself is generated by fluctuations in air pressure. The amplitude of a sound wave corresponds to the level of this pressure at any given instant. We perceive amplitude as loudness, which is commonly measured in decibels (dB). The decibel scale in the real world, known as Sound Pressure Level (SPL), begins at 0 dB, the softest sound a human can perceive. For context, a normal conversation is typically around 60 dB, while a rock concert can reach 125 dB, which is near the threshold of pain for human hearing. However, in the digital domain, the decibel scale is inverted. 0 dBFS (Decibels Full Scale) represents the maximum possible amplitude that can be recorded. All other amplitudes are expressed as negative values. As a general guideline, a decrease of 6 dB equates to halving the signal's amplitude, and any sound below -60 dBFS is typically considered inaudible.

The precision of each audio sample is determined by its **bit depth**. Bit depth refers to the number of bits used to represent the amplitude value of each sample. Since computers operate with a finite set of numbers, the continuous, real-world audio wave must be approximated to the nearest available value in this set. This process is called **quantization**.

Quantization introduces a small amount of error, known as quantization noise. A higher bit depth provides a greater number of possible values, which results in a more accurate representation and, consequently, less quantization noise. While the quantization noise in 16-bit audio is already very low, higher bit depths like 24-bit are standard in professional audio for providing more headroom and an even lower noise floor during processing.

Audio data is stored in quantized formats (like in .wav or .mp3 files), but when working and processing this type of data it's standard practice to convert the audio signal into floating-point numbers to offer a much closer approximation to the continuous values of the original analog signal. The signal is only quantized back to a fixed bit depth when the final audio needs to be saved to a file or sent to a digital-to-analog converter for playback.

Audio Representations and Feature Extraction

A common method for visualizing sound is the waveform, which provides a graphical representation of the amplitude of the audio signal as a function of time. This representation is formally known as the time-domain representation of the signal. In this domain, the vertical axis, or y-axis, corresponds to the amplitude of the signal, while the horizontal axis, or x-axis, represents the passage of time. Consequently, each discrete point on the waveform plot corresponds to a single sample value captured during the digital sampling process of the sound. Examining a waveform helps us understand an audio signal by revealing its critical features. These include the timing of specific sonic events, the overall loudness of the signal, and the presence of any unwanted noise or anomalies.

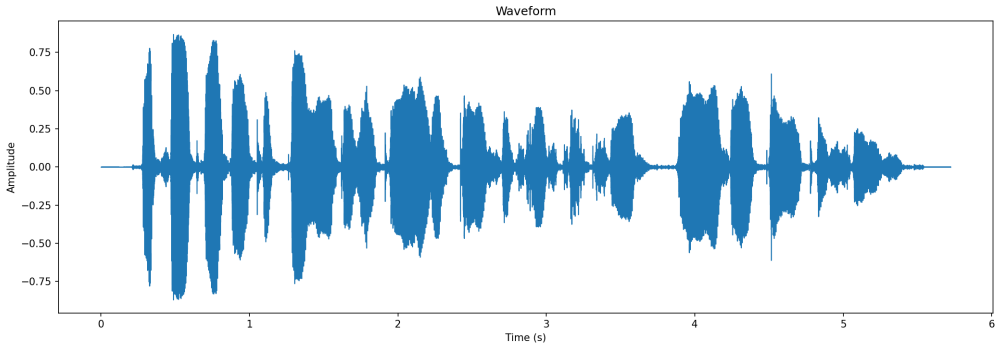


Figure 2.2: Waveform Illustration

An alternative to the time-domain waveform is representing an audio signal in the **frequency domain**. This is accomplished by applying the Discrete Fourier Transform (DFT), often implemented via the Fast Fourier Transform (FFT) algorithm. The frequency domain provides a representation of the signal's composition by illustrating the magnitude of each frequency component. The Discrete Fourier Transform (DFT) is a mathematical technique that converts a finite sequence of discrete data points from the time domain into the frequency domain. It reveals the frequency components present in a signal by decomposing it into a sum of sinusoidal waves with different frequencies, amplitudes, and phases.

This visualization plots the signal's power or magnitude as a function of frequency. The horizontal axis represents frequency, typically on a logarithmic scale to better align with human perception, while the vertical axis denotes the magnitude of each frequency component.

To analyze how the frequency components of an audio signal evolve, a standard spectrum (which provides a single "snapshot" in time) is insufficient. For this task a spectrogram is used. A spectrogram is a time-frequency representation of a signal that visualizes how its spectral content evolves over time. The horizontal axis represents

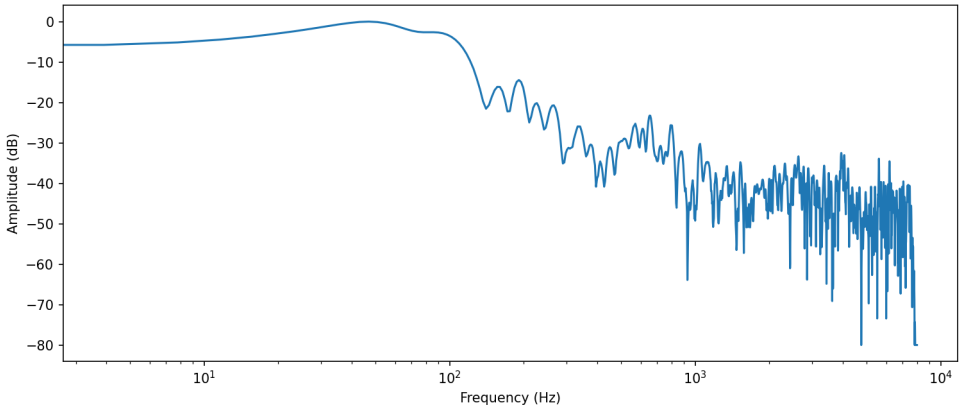


Figure 2.3: Frequency Domain Illustration

time, and the vertical axis represents frequency in Hertz (Hz). The amplitude or power of a frequency component at a specific point in time is encoded by color intensity, typically expressed in decibels (dB).

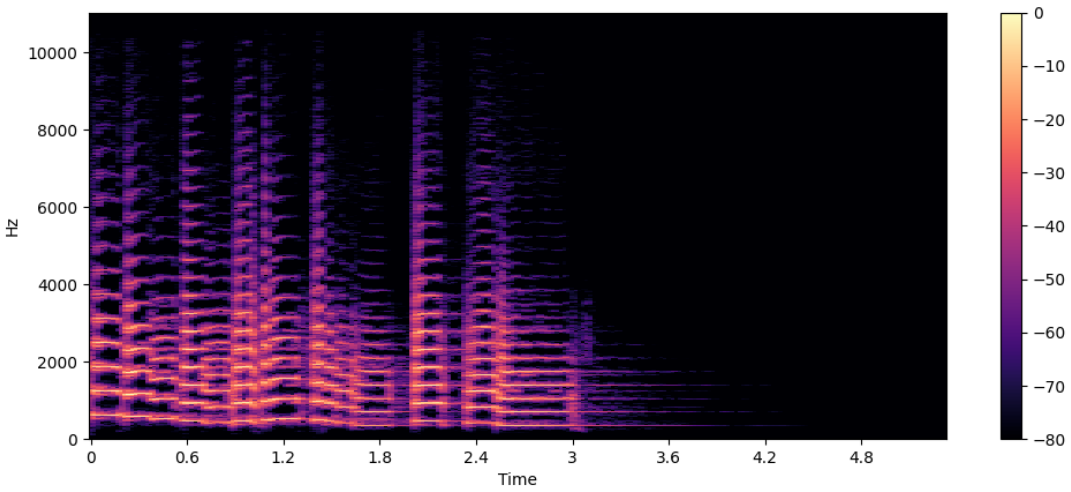


Figure 2.4: Spectrogram Illustration

This visualization is generated using the Short-Time Fourier Transform (STFT). The process involves segmenting the signal into short, overlapping frames (often through a process called windowing) and computing the Discrete Fourier Transform (DFT) for each frame. These resulting frequency spectra are then concatenated along the time axis, where each vertical slice of the spectrogram corresponds to the spectrum of a single time frame.

2.2 Speech Recognition Methods

2.2.1 Hidden Markov Models (HMMs)

Hidden Markov Models (HMMs) revolutionized automatic speech recognition by providing a mathematically principled framework for modeling sequential acoustic data with underlying hidden structure. For decades, HMMs were the dominant approach in ASR, establishing many of the fundamental concepts that persist even in contemporary systems.

A **Hidden Markov Model** is a statistical model in which the system being modeled is assumed to be a Markov process with unobserved (hidden) states. It is often described as a "doubly stochastic process" because it involves two sets of probabilities:

- An underlying, unobservable Markov chain of hidden states.
- A set of probability distributions that link these hidden states to the observable outputs or emissions.

An HMM is formally characterized by the following five elements, collectively denoted by $\lambda = (A, B, \pi)$:

- **N, the number of hidden states:** The set of hidden states is $S = \{S_1, S_2, \ldots, S_N\}$. The state of the system at time t is denoted by $q_t \in S$.
- **M, the number of distinct observation symbols per state:** This is the output alphabet, $V = \{v_1, v_2, \ldots, v_M\}$. The observation at time t is $O_t \in V$. While this definition implies discrete observations, HMMs can also handle continuous observations (e.g., acoustic feature vectors in speech), where the emission probability becomes a probability density function.
- **State Transition Probability Distribution (A):** This is an $N \times N$ matrix $A = \{a_{ij}\}$, where $a_{ij} = P(q_{t+1} = S_j \mid q_t = S_i)$ for $1 \leq i, j \leq N$. This is identical to the transition matrix in a Markov chain and governs the dynamics of the hidden state sequence.
- **Observation (Emission) Probability Distribution (B):** For discrete observations, this is an $N \times M$ matrix $B = \{b_j(k)\}$, where $b_j(k) = P(O_t = v_k \mid q_t = S_j)$ for $1 \leq j \leq N$ and $1 \leq k \leq M$. This represents the probability of observing symbol v_k when the underlying hidden state at time t is S_j. If observations are continuous, $b_j(O_t)$ is a continuous probability density function (e.g., a Gaussian or a Gaussian Mixture Model) defining $P(O_t \mid q_t = S_j)$.

- **Initial State Distribution (π):** This is an N-dimensional vector $\pi = \{\pi_i\}$, where $\pi_i = P(q_1 = S_i)$ for $1 \leq i \leq N$. It specifies the probability of the HMM starting in hidden state S_i.

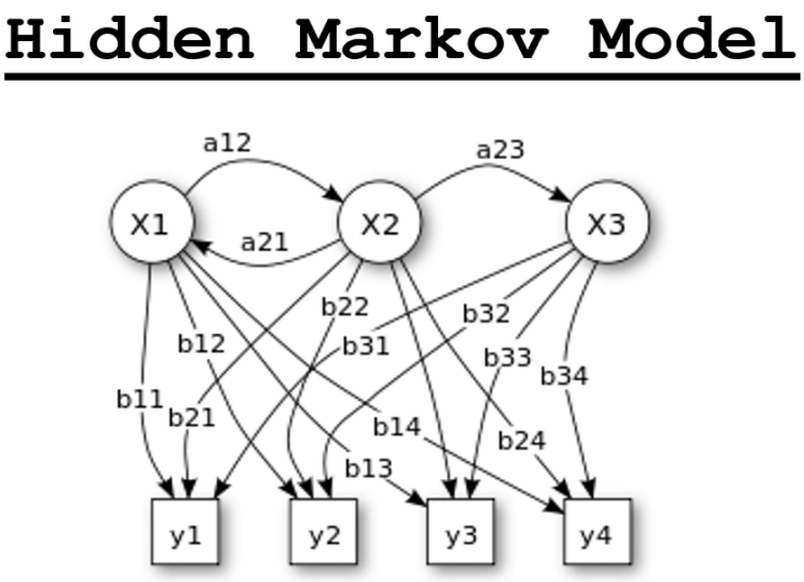


Figure 2.5: Hidden Markov Model [Wis23]

In the context of automatic speech recognition, these abstract components have a concrete physical meaning. HMMs work by representing the fundamental sounds of a language, typically phonemes, as the hidden states (S). The speech is broken down into small sequential chunks of audio features, which are the sequence of observations (O_t). The HMM then calculates the most probable sequence of these hidden phoneme states that would produce the observed audio features. It does this by considering two key probabilities: the likelihood of transitioning from one phoneme to another (for instance, how likely the sound "a" is to follow the sound "c" in "cat"), corresponding to the *transition probabilities* (A), and the probability that a specific acoustic feature vector was generated by a particular phoneme, corresponding to the *emission probabilities* (B).

The early success of HMMs established a paradigm for ASR that involved several key stages: feature extraction from the raw audio, acoustic modeling to relate these features to linguistic units, language modeling, and decoding to determine the most likely sequence of words. Within the HMM framework, various approaches emerged and were continuously refined, including HMM-GMM (Gaussian Mixture Model) systems that modeled acoustic variability, and later HMM-DNN (Deep Neural Network) hybrids that leveraged neural networks for improved acoustic modeling. These variations shared common foundational elements such as Mel-Frequency Cepstral Coefficients (MFCCs) as feature representations, the modeling of phonemes as fundamental

speech units, and sophisticated algorithms like the *Baum-Welch* algorithm for training HMM parameters.

Importantly, all HMM-based methods operate on a generic system architecture with independent, modular components that can be optimized separately while maintaining the overall pipeline structure. Despite the evolution of HMM-based approaches from simple acoustic models to sophisticated hybrid systems, a general structure and pipeline can be described in the majority of HMMs based Speech Recognition Systems. These abstract steps are performed in one way or another by various speech models for transcribing spoken language.

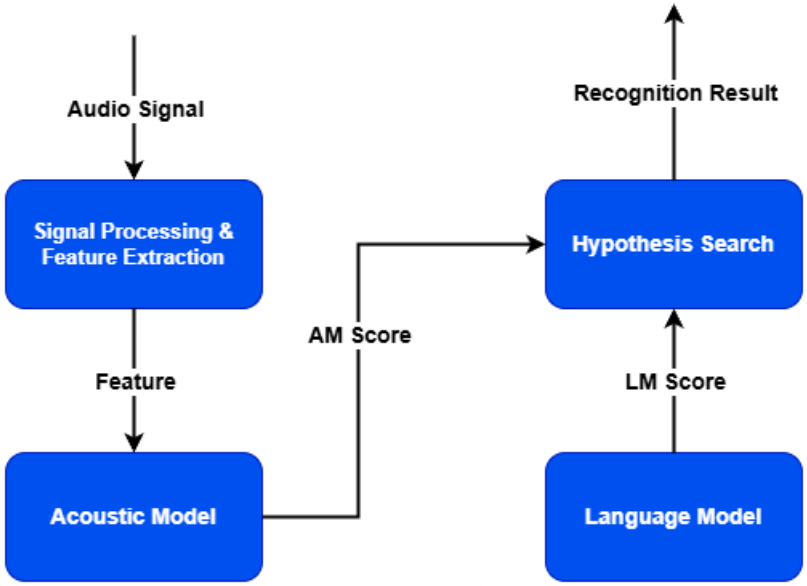


Figure 2.6: General HMM based Speech Recognition Pipeline

The first step in the speech recognition pipeline is the **signal processing and feature extraction component**. This first component takes as input the audio signal, enhances the speech by removing noise and channel distortions, converts the signal into a spectrogram representation describing the frequency domain, and extracts the feature vectors used by the next component of the system, the acoustic model. This first component ensures the processing of the audio input in a way that can be better analyzed by the other components of the system, usually done by calculating Mel frequency cepstral coefficients (MFCCs). This process transforms the spectrogram's raw frequency data onto the perceptually meaningful Mel scale, which mirrors human hearing.

The **Acoustic Model** has the task of learning the relationship between the feature vectors extracted in the previous step and phonetic units such as phonemes to calculate the likelihood that a feature vector corresponds to a specific phonetic unit of the

vocabulary. The output of an acoustic model is usually a probability distribution of the possible phonemes for each given time frame.

The **language mode**l (LM) estimates the probability of a sequence of words, $P(W)$, from a given training text corpus. It captures the syntactic and semantic constraints of a language, helping the ASR system to distinguish between acoustically similar but linguistically different word sequences (e.g., preferring "recognize speech" over the acoustically similar "wreck a nice beach"). N-gram models, which estimate the probability of a word given the previous $n-1$ words, have been commonly used language models in HMM-based ASR systems.

The overall goal in ASR is to find the word sequence $\hat{W}$ that is most probable given the observed acoustic feature sequence X. Using Bayes' theorem, this can be expressed as:

$$\hat{W} = \arg\max_W P(W|X) = \arg\max_W \frac{P(X|W)P(W)}{P(X)}$$

Since $P(X)$ is constant for a given input, this simplifies to:

$$\hat{W} = \arg\max_W P(X|W)P(W)$$

Here, $P(X|W)$ is the acoustic likelihood provided by the HMM-based acoustic model, and $P(W)$ is the language model probability.

The final block, also known as the Decoder, is responsible for the **Hypothesis Search**; it combines the Acoustic Model and Language Model probabilities and outputs the highest probability word sequence as the recognition result. This calculation is performed using an efficient dynamic programming algorithm called *Viterbi Algorithm* that finds the optimal path through the state space by maximizing the joint probability of the observation sequence and the hidden state sequence. At each time step t, for each possible state S, the Viterbi algorithm calculates the maximum probability by considering all possible previous states and selecting the one that, when combined with the transition probability and the emission probability of the HMMs, produces the highest overall probability. By finding the sequence of phonemes with the highest overall probability, the system can map these sounds to words and ultimately recognize what was said.

One of the limitation of these HMMs based models is the scattered modular architecture that requires multiple independent components that must be separately trained and optimized. The lack of end-to-end optimization creates a significant challenge because modifications or improvements to one component operate in isolation from the others, making it difficult to predict whether changes will enhance or degrade overall system performance. Another limitation arises from the inherent Markov assumption that state probabilities at time t depend only on the previous state at time

$t-1$, an assumption that fails to capture the multi-state dependencies characteristic of speech sounds[Rab89].

Despite these limitations and being largely superseded by modern deep learning approaches, HMMs remain fundamental to understanding speech recognition and continue to influence contemporary architectures.

2.2.2 End-to-end (E2E) Deep Learning Based Methods

In the previous chapter, we examined how HMM-based methods require multiple independent modules working together in a pipeline. Each module serves a predefined purpose and handles a specific section of the recognition process. A significant limitation of these traditional approaches is their optimization for very specific, often conflicting criteria. In contrast, End-to-End systems are optimized for a single objective: minimizing Word Error Rate (WER). This chapter explores the alternative approach to speech recognition through E2E systems.

The core idea behind E2E models is to collapse the conventional pipeline - with its distinct AM, LM, and decoding stages - into a single, integrated neural network (usually RNNs, Transformers, or combinations of both). This model directly maps input acoustic features, such as waveforms, spectrograms, or Mel-frequency cepstral coefficients (MFCCs) to individual characters or word sequences.

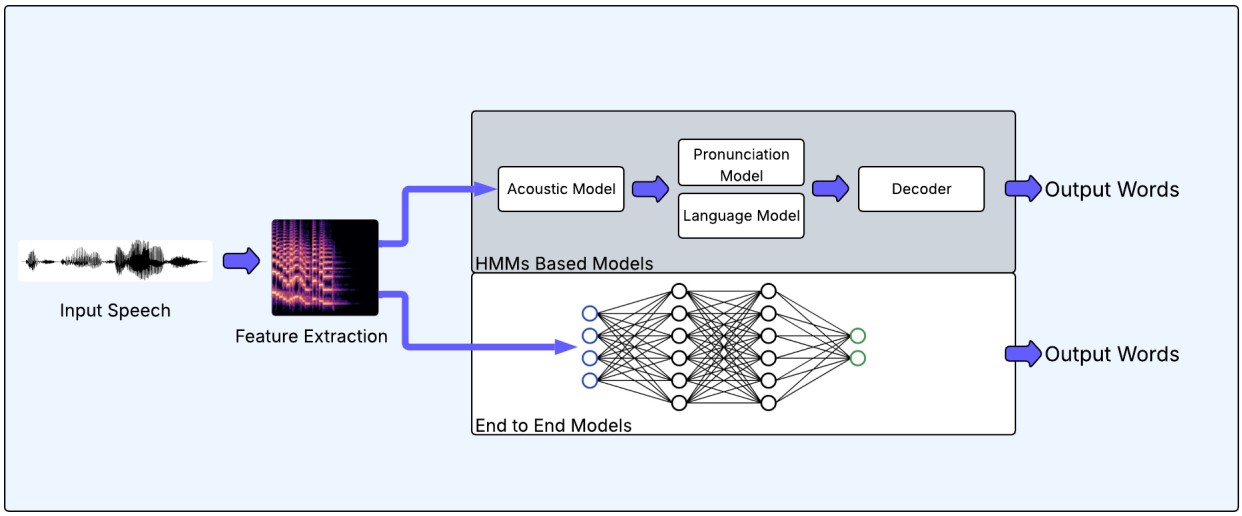


Figure 2.7: HMMs vs End to end models comparison

One of the main motivations for switching to end-to-end (E2E) models in speech recognition is simplification. Traditional systems rely on several hand-crafted components like acoustic models, language models, and phoneme dictionaries. E2E models replace these with a single deep neural network, reducing system complexity and often lowering memory usage. Another benefit is that we no longer need to rely on

assumptions about intermediate units like phonemes or triphones. Instead, the neural network learns directly from the raw audio features, extracting useful and more complex patterns on its own. Finally, E2E models allow training the entire system jointly, which can lead to better optimization and improved performance.

The training process usually optimizes a single objective function, such as cross-entropy loss or CTC loss, depending on the model type. Then, the backpropagation algorithm updates the model's weights and biases based on the loss value and the learning rate parameter. The number of training samples used in each learning step is defined by the batch size. This process is repeated over the entire dataset for several epochs or until a stopping condition is met.

If not done properly, training can lead to overfitting, where the model memorizes patterns specific to the training data and performs poorly on new data. To prevent this, techniques such as regularization, which limits large model weights, or dropout, which randomly deactivates neurons during training, can be used.

Training end-to-end systems effectively requires significant computing power and large amounts of labeled data. This means using powerful hardware like GPUs and accessing large-scale datasets to reduce overfitting and improve generalization.

There are several types of E2E architectures currently in use, each employing different mechanisms to align input sequences with output text. The most prominent include:

- **Connectionist Temporal Classification (CTC):** This was the first proposed end-to-end speech recognition method. It shares a key idea with traditional HMMs by treating each part of the audio signal separately, assuming the prediction at one moment doesn't depend on the prediction from the moment before[GFGS06].

- **Sequence to Sequence (Seq2Seq2):** These models use attention mechanisms to dynamically focus on different parts of the input sequence during decoding, enabling the model to capture long-range dependencies and contextual information more effectively[CJLV15].

Connectionist Temporal Classification (CTC) Methods

Connectionist Temporal Classification (CTC) is a powerful method for sequence-to-sequence tasks where the alignment between the input and output is not known, a common challenge in speech recognition. Introduced by [GFGS06], its main advantage is that it eliminates the need for pre-aligned data, allowing the model to learn how to map a long input sequence, like audio features, to a much shorter output sequence, such as text, directly.

At its core, CTC works by first having a neural network (often a Recurrent Neural Network like an LSTM) predict a probability distribution over all possible output labels for each timestep or frame of the input audio. To handle the fact that spoken words have varying lengths and that there are often silent or non-alphabetic segments, CTC introduces a special **'blank'** token: **'_' or `<blank>`**. This token represents a "no output" prediction. The network, therefore, predicts a character from the vocabulary or this special `<blank>` token at every single input step.

The process of generating the final output from this frame-by-frame prediction is a two-step collapsing mechanism. First, any repeated labels in the sequence are merged into a single label. Second, all `<blank>` tokens are removed. This elegant process allows the model to produce a final transcription of any length. For instance, a sequence of predictions like "`___hhh_eellll_ll_o_`" would be collapsed to the final output "`hello`".

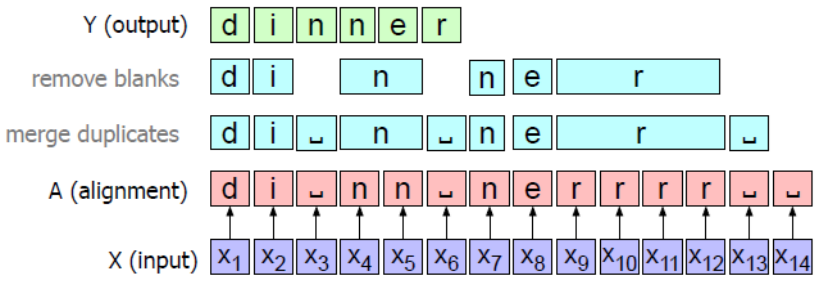


Figure 2.8: CTC processing pipeline [JM25]

A significant challenge in training a CTC model is that a single correct output can be represented by a vast number of valid frame-level prediction sequences. For example, "`go`" could be represented by "`gg_oo`", "`_ggooo`", and many other variations. To address this during training, the **CTC loss function** is designed to efficiently sum the probabilities of all the valid alignments that correspond to the correct output sequence. This is typically achieved using a dynamic programming approach known as the **forward-backward algorithm**, which allows for the efficient calculation of this sum. The model is then trained by minimizing the negative log-likelihood of the correct output sequence given the input:

$$\mathcal{L}_{\text{CTC}} = -\log P(Y|X) = -\log \sum_{A \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} p(a_t|X) \tag{2.1}$$

where A is a possible alignment, $\mathcal{B}$ is the collapsing function, and $p(a_t|X)$ is the probability of a specific label at time t.

For decoding, a simple and fast approach is **greedy decoding**. At each timestep, it selects the most probable token (character or `<blank>`). The resulting sequence of most likely tokens is then collapsed to produce the final output. While efficient, this method is not optimal because the most probable path of alignments does not necessarily lead to the most probable final sequence. A more sophisticated method, **beam search**, is often used to achieve better results by keeping track of a number of the most likely alignment hypotheses at each step.

A key architectural aspect of CTC is the **conditional independence assumption**. It assumes that the prediction at any given timestep is independent of the predictions at other timesteps, given the input audio. This is a strong simplification, as the letters in a word are clearly not independent of each other. This is also CTC's main limitation, as it lacks an inherent language model to understand the context between output tokens. This can lead to spelling errors that a language model would typically prevent, such as confusing "two" with "to" or producing phonetically plausible but nonsensical words. Despite this drawback, CTC's simplicity, efficiency, and its ability to be paired with external language models make it a widely used and effective technique in modern speech recognition systems.

Sequence to Sequence (Seq2Seq) Methods

The Sequence to Sequence method achieved impressive results in tasks such as machine translation, text summarization, and speech recognition. It was first introduced by [SVL14] and it's power lies in its ability to first read and understand the entire input sequence and then generate a new output sequence based on that understanding. This technique has 2 main components: the encoder and the decoder.

The **encoder**'s job is to process the entire input sequence (e.g., audio data) and compress it into a fixed-size numerical representation. It processes the input step-by-step and produces a sequence of "hidden states" which capture information from the input. The final representation, often called the **context vector**, is a summary of the entire input sequence.

The **decoder** takes the context vector from the encoder and begins generating the output sequence element by element. It is **autoregressive**, meaning that to generate the next element in the sequence, it considers not only the encoder's context vector but also the elements it has already generated. This process continues until the decoder

produces a special end-of-sequence token.

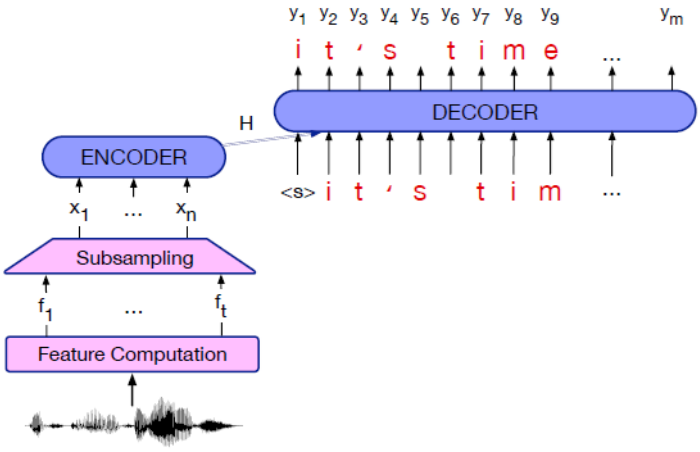


Figure 2.9: Encoder Decoder Speech Recognition [JM25]

In the original Seq2Seq model[SVL14], the decoder relied solely on the *final* hidden state from the encoder. This single context vector had to encapsulate the meaning of the entire input sequence. For short inputs, this worked reasonably well. However, for long sequences, this design created a bottleneck. It became difficult for a single vector to retain all the necessary details from a lengthy input, leading to a loss of information and a decrease in performance.

To solve the bottleneck problem, the **attention mechanism** was proposed [CJLV15]. Attention provides a way for the decoder to look back at the *entire* sequence of the encoder's hidden states at every step of the output generation process. Instead of forcing the model to rely on a single context vector, attention allows the decoder to "attend to" the most relevant parts of the input sequence when generating a specific part of the output.

The core idea is to replace the single, static context vector with a **dynamic context vector** that changes for each output step. At each decoding step, the decoder calculates a set of **attention scores**. These scores measure how relevant each of the encoder's hidden states is for generating the current output element. These scores are then converted into **attention weights** (typically using a softmax function), which represent the importance of each input part. A new, dynamic context vector is created by taking a **weighted sum** of all the encoder's hidden states. The weights ensure that the parts of the input sequence with higher importance contribute more to this context vector. This dynamic context vector is then used by the decoder, along with its previous output, to generate the next element. This process allows the decoder to focus on different parts of the input at each step, overcoming the information bottleneck and dramatically improving performance, especially on long sequences.

Encoder-decoders for speech are trained with the normal **cross-entropy** loss. At

timestep i of decoding, the loss is the log probability of the correct token (letter) y_i. The loss for the entire sentence is the sum of these losses:

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{m} \log p(y_i \mid y_1, \ldots, y_{i-1}, X) \tag{2.2}$$

This loss is then backpropagated through the entire end-to-end model to train the entire encoder-decoder.

The encoder-decoder framework is especially well suited for tasks where the input and output sequences differ greatly in length, such as in speech processing. In this context, lengthy sequences of acoustic features must be translated into relatively short sequences of words or characters. For example, a single word might consist of just three letters, yet span around 100 acoustic frames if it lasts one second (assuming a frame every 10 milliseconds). This approach does not rely on a strict one-to-one alignment between input and output elements, allowing for flexibility in sequence lengths. This stands in contrast to CTC-based decoding, which requires generating an output for each input timestep, followed by a post-processing step to merge and simplify those predictions.

2.3 Evaluation Metrics

In order to assess the performance of automatic speech recognition (ASR) systems, it is essential to use quantitative metrics that measure how closely the system's output matches a reference transcription. Word error rate (WER) is the standard metric used to evaluate speech recognition systems. It measures the difference between the recognizer's output (the hypothesized word string) and the reference transcription.

The formula for Word Error Rate is the following:

$$\text{WER} = \frac{S + D + I}{N}$$

Where:

- **S** is the number of **substitutions**. A substitution error occurs when a word in the reference is replaced with a different word in the hypothesis (e.g., ”this” is transcribed as ”is”).
- **D** is the number of **deletions**. A deletion error occurs when a word present in the reference is missing from the hypothesis.
- **I** is the number of **insertions**. An insertion error occurs when a word is present in the hypothesis but not in the reference.

- **N** is the total number of words in the **reference** transcript.

Because of insertion errors, the WER can exceed 100%. A lower WER indicates higher accuracy, with a WER of 0 representing a perfect transcription.

Character Error Rate (CER) is a similar metric, but it operates at the character level instead of words. This makes CER useful for evaluating models in domains where precision at the letter level is important, such as in medical terminology. CER is calculated using the same principle:

$$\text{CER} = \frac{S + D + I}{N}$$

Here, N represents the number of characters in the reference transcription.

Both WER and CER are based on the *Levenshtein distance*, which measures the minimum number of edits: insertions, deletions, and substitutions required to transform the hypothesis into the reference.

Chapter 3

Implementation of a Medical Domain ASR System

This chapter presents the practical application of my thesis: the development and deployment of an enhanced speech recognition system tailored for medical environments. The first part details the complete machine learning pipeline, from implementing LoRA fine-tuning on the Whisper architecture to deploying the improved model via RESTful API. The second part introduces medspeech.io, a web application that integrates our enhanced model to enable physicians to transcribe patient conversations and manage medical documentation efficiently.

3.1 Fine-Tuning an ASR model on medical data

Building on the theoretical foundation established in the previous chapter, the objective is to develop a machine learning model capable of accurately transcribing speech within the medical domain—a context where precision is critical and domain-specific terminology is frequently used.

We begin by defining the problem and the specific requirements of medical transcription. Next, we select a suitable base model—OpenAI's Whisper—and justify this choice in light of its architecture and performance characteristics. We then describe the acquisition and structure of the training data, along with the preprocessing steps required to prepare it for fine-tuning.

To adapt the pretrained model to the medical domain, we apply a parameter-efficient fine-tuning strategy called Low-Rank Adaptation (LoRA). We then evaluate the fine-tuned model using relevant metrics.

3.1.1 Problem Definition

While modern ASR systems have reached high levels of accuracy on general-purpose benchmarks, their performance drops significantly when applied to domain-specific contexts such as medicine. In clinical environments, speech often contains highly specialized vocabulary, uncommon terms, acronyms, and abbreviations that are absent from general training corpora. Additional complications arise from diverse speaker accents, spontaneous speech patterns, and noisy environments—conditions typical in real-world medical settings. This mismatch between general ASR capabilities and the linguistic characteristics of medical speech highlights the need for specialized models. Therefore, the core problem addressed in this work is how to adapt an existing ASR system to perform reliably in the medical domain by bridging the gap between it's general pretrained knowledge and domain-specific linguistic requirements.

3.1.2 Choosing a base ASR model

The selection of an appropriate baseline pre-trained model is a critical first step in developing a specialized and robust Automatic Speech Recognition (ASR) system. The ideal candidate must be accessible for research, adaptable to new domains, and demonstrate good performance. The primary criterion for this project is transcription accuracy, a domain where Transformer-based architectures have consistently defined the cutting edge. Also, as we saw in the previous chapter, the Encoder-Decoder architecture is well suited for the sequence-to-sequence nature of ASR tasks. Therefore, our evaluation focuses on leading open-source models that leverage this type of architecture.

In recent years, the field of speech recognition has been dominated by large-scale, pre-trained models. A good example is wav2vec 2.0[BZMA20], a framework that learns powerful speech representations directly from raw audio through a self-supervised, contrastive learning task. While highly effective, its primary output is a robust audio encoder, which still necessitates a purpose-built decoder to become a full transcription system.

Other noteworthy open-source contributions, such as those from the NVIDIA NeMo or ESPnet toolkits, have advanced the field by implementing novel architectures like the Conformer, which combines convolutional layers with Transformers to capture both local and global dependencies in the audio signal.

After a thorough review of the available open-source models, **OpenAI's Whisper**[RKX+23] was selected as the baseline for this project based on several key architectural and training advantages that make it a good fit for adaptation to a specialized domain like medicine:

1. **Transformer-Based, End-to-End Architecture:** At its core, Whisper is an encoder-decoder Transformer model.

2. **Large-Scale, Weakly-Supervised Pre-training:** Whisper's most significant differentiator is its pre-training regimen. It was trained on an unprecedented 680,000 hours of audio from the internet, which was "weakly supervised" by its accompanying text. This massive and incredibly diverse dataset, covering a wide range of speakers, environments, topics, and languages, has endowed the model with an exceptional degree of generalization and robustness.

3. **Superior Zero-Shot Performance:** A direct consequence of its training is Whisper's remarkable zero-shot capability.

4. **Open Source and Accessible:** The Whisper models and the corresponding inference code have been made publicly available by OpenAI.

Dataset	wav2vec 2.0 Large (no LM)	Whisper Large V2	RER (%)
LibriSpeech Clean	**2.7**	**2.7**	0.0
Artie	24.5	**6.2**	74.7
Common Voice	29.9	**9.0**	69.9
Fleurs En	14.6	**4.4**	69.9
Tedlium	10.5	**4.0**	61.9
CHiME6	65.8	**25.5**	61.2
VoxPopuli En	17.9	**7.3**	59.2
CORAAL	35.6	**16.2**	54.5
AMI IHM	37.0	**16.9**	54.3
Switchboard	28.3	**13.8**	51.2
CallHome	34.8	**17.6**	49.4
WSJ	7.7	**3.9**	49.4
AMI SDM1	67.6	**36.4**	46.2
LibriSpeech Other	6.2	**5.2**	16.1
Average	29.3	**12.8**	55.2

Figure 3.1: Relative Error Reduction[RKX+23]

Architecture

The core of Whisper adheres to the standard Transformer architecture3.2, comprising two main components: an encoder and a decoder.

Encoder

The encoder's role is to process the input log-Mel spectrogram and transform it into a sequence of continuous representations. It is composed of a stack of identical layers. Each encoder layer typically contains two primary sub-layers: a multi-head self-attention mechanism, which allows the model to weigh the importance of different

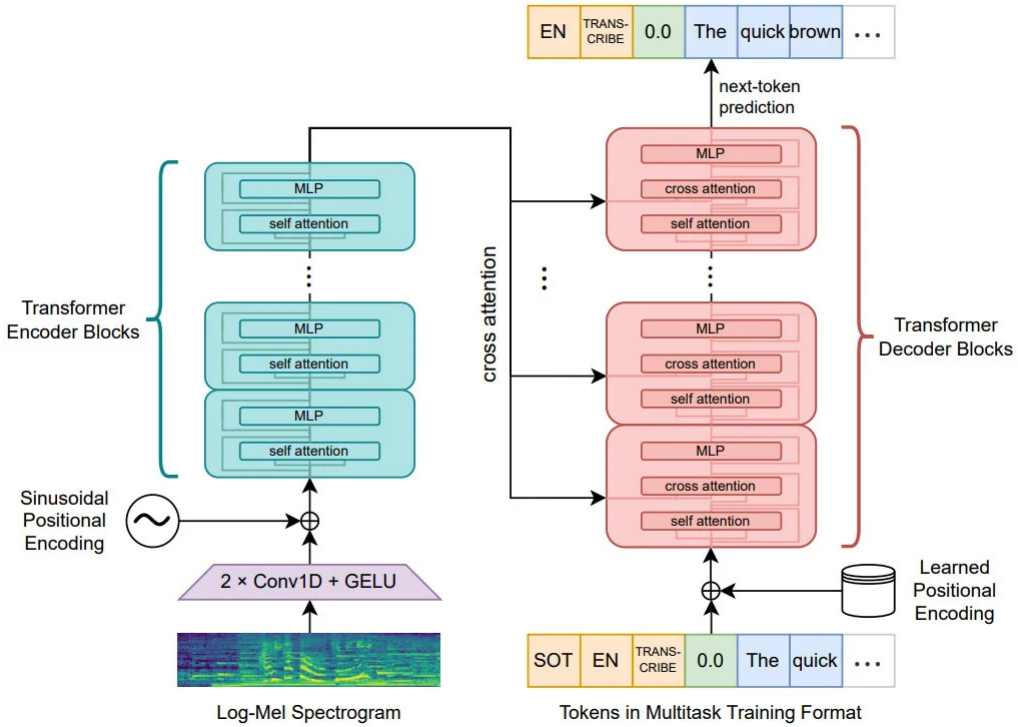


Figure 3.2: Whisper overview

parts of the input sequence relative to each other, and a position-wise fully connected feed-forward network, which applies a non-linear transformation independently to each position. Residual connections are employed around each of these two sub-layers, followed by layer normalization, to facilitate gradient flow and stabilize training.

Decoder

The decoder is also structured as a stack of identical layers. It takes the encoder's output and the previously generated text tokens as input to produce the next token in the output sequence. In addition to the two sub-layers found in each encoder layer (multi-head self-attention and feed-forward network), the decoder inserts a third sub-layer. This additional sub-layer performs multi-head cross-attention over the output of the encoder stack, enabling the decoder to focus on relevant parts of the input audio sequence while generating the transcript. The self-attention sub-layer within the decoder is "masked" to prevent positions from attending to subsequent positions. This masking is crucial for maintaining the autoregressive property of the decoder, ensuring that the prediction of a token only depends on previously generated tokens and not future ones.

Input Audio Preprocessing

Starting with audio preprocessing, as it's described in the Whisper paper[RKX+23], the model first resamples the input audio to 16kHz. This is pretty common in machine learning audio models since 16kHz is enough to capture human speech clearly.

After that, the audio gets turned into a 2D log-mel spectrogram with 80 mel bands. To do this, it first normalizes the raw audio to values between -1 and 1, then applies a Short-Time Fourier Transform using a sliding window: 25 milliseconds per window, with a 10 ms stride. This gives a standard spectrogram. The STFT is done using the *librosa* Python library.

Then, it multiplies the spectrogram by mel filter banks with 80 bands to shift the frequency information from Hz to mels. In the end, we get a PyTorch tensor with a `[80, T]` shape, where 80 is the number of mel bins, and T is the time dimension (the number of frames we get after applying those 25 ms windows with 10 ms steps).

Tokenizer

Whisper processes text data using a subword tokenization scheme based on Byte-Pair Encoding (BPE).

- **Byte-Pair Encoding (BPE):** BPE is a data compression algorithm that iteratively merges the most frequent pair of bytes (or characters) in the training corpus to form new subword units. This allows the model to handle a large vocabulary, including rare words and out-of-vocabulary words, by representing them as sequences of common subword tokens.

The Whisper architecture employs a symmetrical design where, for any given model variant, the encoder and decoder possess an equal number of Transformer blocks (layers).

The primary activation function explicitly mentioned for specific components in the Whisper architecture is the **Gaussian Error Linear Unit (GELU)**.

A key architectural innovation of Whisper is its unified multitask output format 3.3, which allows a single model to perform various speech-related tasks by predicting a sequence of special tokens in addition to the transcribed text. This approach simplifies the overall system by eliminating the need for separate models for each task.

The decoder is conditioned on, and trained to predict, a set of special tokens that define the task and control the output format:

- `<|startoftranscript|>`: This token signals the beginning of the transcript prediction.

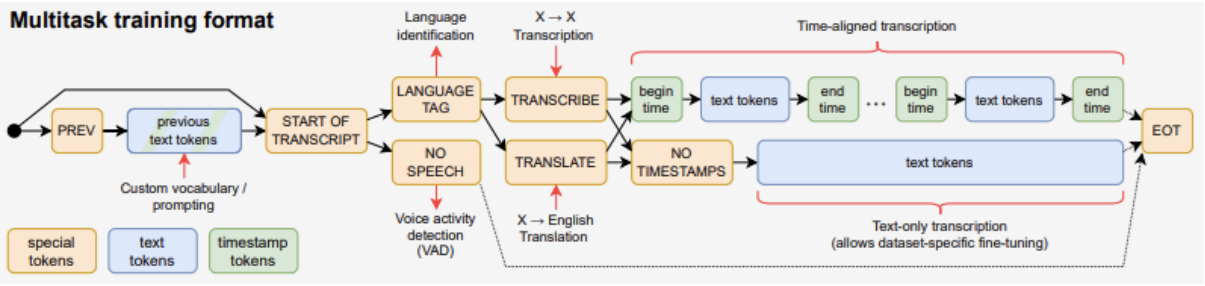


Figure 3.3: Whisper Output Tokens

- **Language Tokens**: Immediately following `<|startoftranscript|>`, the model predicts a language token (e.g., `<|en|>` for English, `<|fr|>` for French) corresponding to the detected language of the audio segment. Whisper supports 99 unique language tokens, with the language targets during training being sourced from an auxiliary language identification model fine-tuned on VoxLingua107.
- `<|nospeech|>`: If the model detects no speech in an audio segment, it is trained to predict this token, effectively performing Voice Activity Detection (VAD).
- **Task Tokens**: After language identification, a task token specifies the desired operation: `<|transcribe|>` for standard speech-to-text in the identified language, or `<|translate|>` for translating the spoken audio into English text.
- **Timestamp Control Tokens**:
  - `<|notimestamps|>`: If this token is provided or predicted, the model outputs the transcript without any timestamp information.
  - If timestamps are desired, this token is omitted. The model then predicts timestamps relative to the current 30-second audio segment. These timestamps are quantized to the nearest 20 milliseconds, matching Whisper's native time resolution. Special tokens representing these quantized time offsets (e.g., `<|0.00s|>`, `<|0.02s|>`, ..., `<|30.00s|>`) are added to the vocabulary. These timestamp tokens are interleaved with the text tokens: a start-time token is predicted before a segment of text, and an end-time token is predicted after it.
- `<|endoftranscript|>`: This token marks the end of the generated transcript for the current segment.

3.1.3 Data aquisition and preparation

The process of training or fine-tuning deep learning models, begins with the acquisition of high-quality and diverse datasets. The performance of such models—especially

their generalizability and applicability in real-world scenarios—is closely tied to the quality and representativeness of the data on which they are trained.

However, obtaining suitable datasets for medical speech recognition tasks presents significant challenges since medical data is subject to strict regulations (GDPR, HIPAA). These regulations severely limit the public availability of patient-related speech data, making it difficult to obtain appropriate data sets for research and development purposes.

In addition to regulatory constraints, the linguistic and acoustic diversity of the data is another important consideration. For this research, we focus on English-language speech-to-text data, since English is widely used in medical documentation and comparatively easier to source in public datasets. An ideal dataset for this task would include recordings from both male and female speakers and represent a range of English accents to improve the model's robustness. Each data sample should consist of an audio recording accompanied by a precise transcription of the spoken content, ensuring accurate alignment between the speech signal and its textual representation.

A comprehensive review of publicly available datasets revealed two primary resources suitable for medical speech recognition research, each offering distinct advantages and limitations for fine-tuning speech-to-text models in healthcare contexts.

- **Primock57 Dataset:[PKMSS22]** This dataset comprises 57 simulated medical primary care consultations conducted over a five-day period. These consultations involved seven Babylon Health clinicians and 57 Babylon employees who assumed patient roles using structured case cards containing predetermined presenting complaints, symptoms, medical histories, and general background information. The dataset provides a multi-modal approach to medical conversation analysis, including audio recordings of complete consultations, manual utterance-level transcriptions, consultation notes authored by participating clinicians, and comprehensive human evaluation annotations with associated metadata.

- **Syn-Med Dataset:[Car24]** The Syn-Med dataset, introduced in the United-Syn-Med research paper[], employs a novel synthetic data generation approach to address the scarcity of medical speech data. Rather than utilizing recordings of actual medical professionals, the researchers systematically extracted medical terminology and clinical descriptions from authoritative medical references, including ICD-10 disease classification codes, MIMS pharmaceutical databases, and FDA regulatory documentation. These curated medical concepts were subsequently processed through GPT-based language models to generate contextually appropriate medical sentences, which were then converted to speech using StyleTTS 2, an advanced text-to-speech synthesis system. The resulting dataset

encompasses approximately 400,000 unique medical sentences transformed into 790,000 audio recordings, totaling 5,486 hours of synthetic speech data.

Following comparative analysis of both datasets, the Syn-Med dataset was selected for this research due to its comprehensive coverage of medical terminology across multiple healthcare domains. The dataset encompasses diverse medical content categories, including pathological conditions, clinical symptoms, and health-related presentations spanning all medical specialties. Additionally, it incorporates extensive pharmaceutical information, including medication nomenclature, dosage specifications, and therapeutic interventions, thereby covering the pharmacological dimensions of medical practice. The inclusion of regulatory and compliance terminology from FDA databases further extends the dataset's applicability to clinical research contexts, drug approval processes, and pharmaceutical safety protocols, making it a more comprehensive resource for training robust medical speech recognition systems.

Data Collection

The Syn Med dataset, selected for this research, is publicly accessible via the Hugging Face platform. In its entirety, this dataset comprises approximately 5,486 hours of audio, presenting a substantial computational load in terms of processing power and memory requirements. Due to these constraints, a representative subset corresponding to **10% of the total dataset**, amounting to 79,000 audio recordings paired with their corresponding transcriptions, was utilized for this study.

Data Analysis and Visualization

This section presents a comprehensive analysis of the medical speech dataset used for training the Whisper model. The analysis covers various aspects of the audio files including technical specifications, temporal characteristics, and gender distribution.

The basic characteristics of the audio files are presented in Figures 3.4 and 3.5. File sizes follow a normal distribution with most files ranging between 0.04 and 0.06 MB, peaking around 0.05 MB. The duration analysis reveals speech segments spanning 3 to 10 seconds, with optimal concentration between 5 and 7 seconds. This temporal range effectively captures medical terminology and clinical phrases while maintaining computational efficiency.

Bitrate Analysis

Figure 3.6 presents the distribution of MP3 bitrates across the dataset. Bitrate represents the amount of audio data processed per unit of time, measured in kilobits per second (kbps). Higher bitrates generally indicate better audio quality but result in

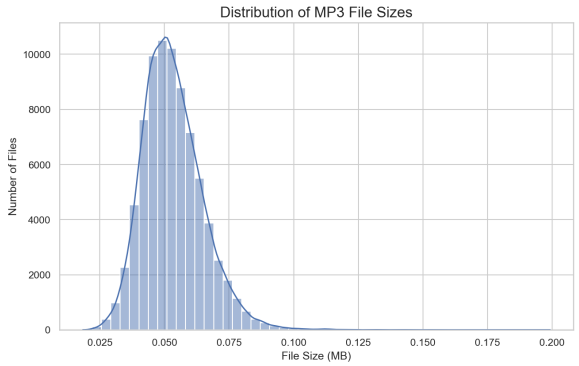


Figure 3.4: Distribution of MP3 file sizes

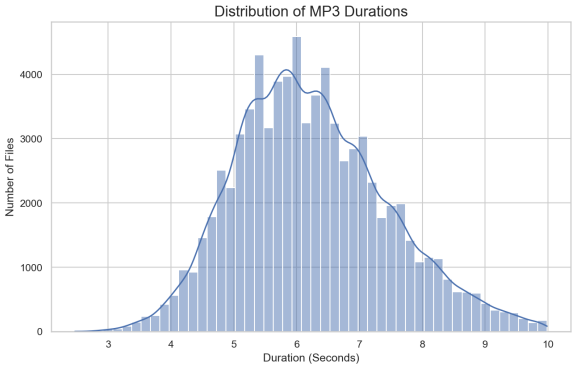


Figure 3.5: Distribution of audio duration in seconds

larger file sizes. The dataset shows a concentration around 70 kbps, which represents a balance between audio quality and file size efficiency. The distribution includes bitrates ranging from 63 to 77 kbps, with 70 kbps being the most common encoding setting.

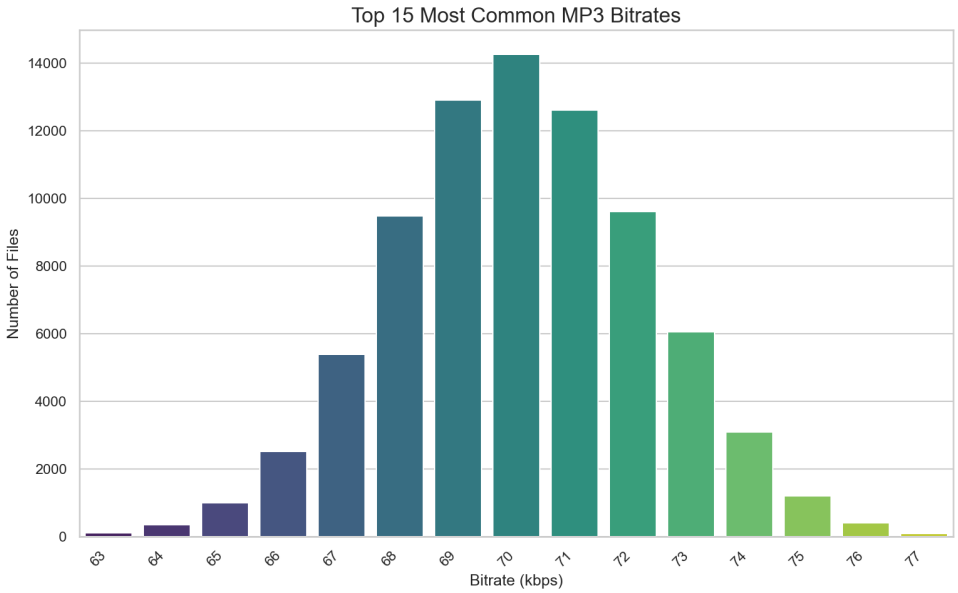


Figure 3.6: Distribution of MP3 bitrates in the medical speech dataset

The demographic composition and speech patterns by gender are illustrated in Figures 3.7 and 3.8. The dataset demonstrates excellent gender balance with 50.1% male speakers (39,636 files) and 49.9% female speakers (39,431 files). Word count analysis reveals similar speech patterns across genders, with both groups showing median values around 13 words per segment and comparable interquartile ranges. Both distributions contain outliers extending up to 35 words, indicating occasional longer medical explanations.

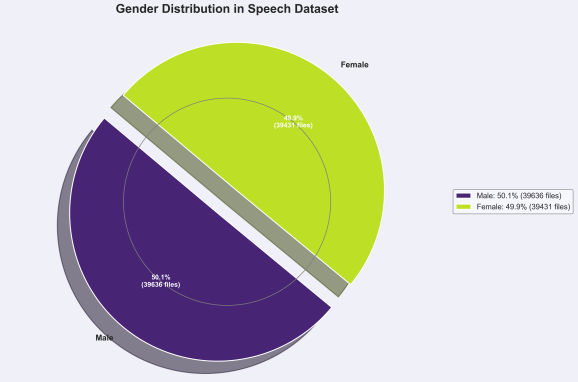


Figure 3.7: Gender distribution in the dataset

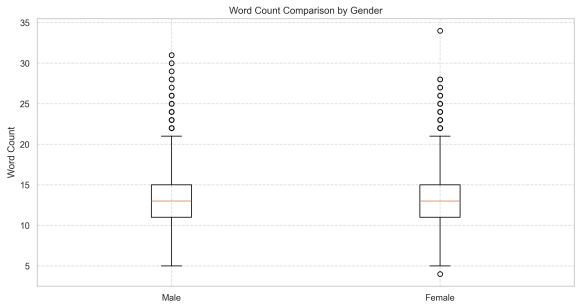


Figure 3.8: Word count comparison by gender

Peprocessing

The data preprocessing pipeline started with an initial **shuffling** of this selected subset to ensure data randomness. Subsequently, each audio sample underwent **resampling to a 16kHz sampling rate** utilizing the `librosa` Python library. This step was necessary as the Whisper model requires audio input at this specific sampling frequency. Following resampling, the dataset was partitioned into three distinct sets: a **training set (80%)** , a **validation set (10%)**, and a **test set (10%)**. The training set was used for model learning, the validation set for hyperparameter tuning and monitoring performance during the training phase, and the test set was reserved for the final, unbiased evaluation of the fine-tuned model's accuracy.

3.1.4 Fine-Tuning Strategy: Low Rank Adaptation (LoRA)

One major drawback of traditional fine-tuning for large-scale machine learning models is the vast number of parameters that must be updated. Transformer-based architectures, while delivering impressive performance across a wide range of tasks, often consist of billions of parameters, making fine-tuning both computationally demanding and resource-intensive. Our chosen model, Whisper (large version), contains 0.8 billion parameters—placing us in a similar situation. Performing full fine-tuning would require substantial computational power and a massive dataset, all for potentially marginal performance gains. Unfortunately, we currently lack both the necessary hardware and the data to pursue this approach.

To address this limitation, we will adopt a more efficient approach to parameter fine-tuning known as Low-Rank Adaptation (LoRA), introduced by [HSW+22]. The key concept behind this fine-tuning approach is to leave the model's original weights unchanged ("frozen") throughout training. Instead of directly updating these weights, we insert small, trainable modules called *adapters* into selected layers of the model.

These adapters consist of two low-rank matrices designed to approximate the updates that would otherwise be applied to the original weight matrix. Specifically, the weight update ΔW is approximated as the product of these matrices:

$$\Delta W \approx A \cdot B$$

The optimization comes from the idea that we are not training the entire weight matrix W, but only the small adapter matrices A and B. We can better quantify this with a mathematical example.

Suppose we have a weight matrix $W \in \mathbb{R}^{n \times m}$. For simplicity, let's assume $n = m = 1000$. This means that the weight matrix W has $1000 \times 1000 = 1,000,000$ parameters.

Now, let's consider the two adapter matrices: $A \in \mathbb{R}^{n \times r}$ and $B \in \mathbb{R}^{r \times m}$, where the rank r is much smaller than the minimum of n and m (i.e., $r \ll \min(n, m)$). If we multiply these two matrices (A×B), we get a matrix of the exact same size as W.

Let's arbitrarily choose a rank $r = 16$. If we count the number of parameters we need to train for the adapters, the result is: $(n \times r) + (r \times m) = (1000 \times 16) + (16 \times 1000) = 16,000 + 16,000 = 32,000$.

This is significantly smaller than the 1,000,000 parameters of the original matrix W.

As a general rule, the number of trainable parameters in LoRA is:

$$r \cdot n + r \cdot m = r(n + m)$$

This value is much smaller than the total number of parameters in the original matrix, which is $n \cdot m$, especially for a small rank r.

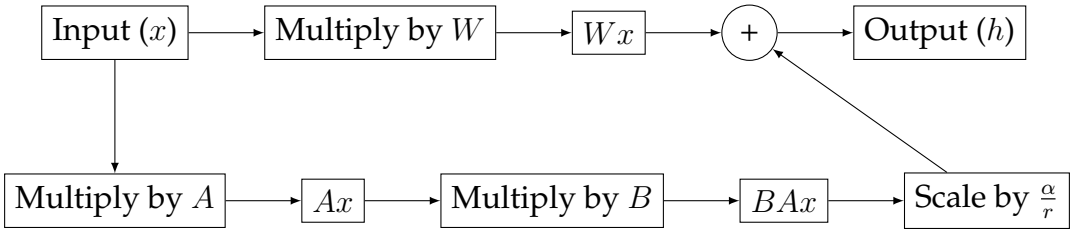
LoRA also introduces a scaling factor, α, which modifies the forward pass to:

$$h = Wx + \frac{\alpha}{r}(BA)x$$

Where:

- W is the original, frozen weight matrix.
- x is the input vector.
- h is the final output of the layer.
- Wx corresponds with the output from the original model path.
- $(BA)x$ is the output from the LoRA adapter path.

Conceptual Diagram of LoRA Integration



Another advantage of LoRA is its mitigation of "catastrophic forgetting." By keeping the original weights (W) frozen, the model's prior knowledge is preserved. This approach is well-suited for our use case, as the base Whisper model already performs well on general data. It has learned fundamental concepts about language and audio that we want to ensure are not forgotten during specialization.

3.1.5 Experimental Setup and Hyperparameters

The model training was conducted on a Google Colab notebook equipped with an NVIDIA A100 GPU with 40GB of VRAM. To facilitate our research, we fine-tuned two distinct models from the Whisper family: the **Whisper large-v3-turbo**, which comprises 0.8 billion parameters, and the **Whisper base**, with 74 million parameters. The training process was monitored using the Weights and Biases platform. Upon completion of the training, the resulting LoRa adapters were saved and uploaded to the Hugging Face Hub. For efficient integration and training of the LoRa adapters, we utilized the Hugging Face PEFT (Parameter-Efficient Fine-Tuning) library.

Training Hyperparameters

The following hyperparameters were employed for the training process:

- **Batch Size:** A batch size of 8 was selected. This was the maximum value achievable without encountering CUDA out-of-memory errors, given the available hardware. The batch size dictates the number of training samples utilized in a single forward and backward pass.
- **Gradient Accumulation Steps:** To compensate for the relatively small batch size, we set the gradient accumulation steps to 4. This technique involves accumulating gradients over four steps and performing a weight update only on the fourth step. Consequently, this simulates a larger effective batch size of 32 (8×4), which can lead to more stable training.
- **Number of Epochs:** The models were trained for a total of 4 epochs.

- **Learning Rate:** We utilized a learning rate of 1×10^{-5} accompanied by 500 warmup steps. During this warmup phase, the learning rate is gradually increased from a lower value to the target rate.
- **Early Stopping:** To mitigate the risk of overfitting, an early stopping mechanism was implemented with a patience value of 3. This means that the training process ends if the validation Word Error Rate (WER) does not show improvements for three consecutive epochs.

LoRa Configuration

For the LoRa (Low-Rank Adaptation) specific hyperparameters, the following configuration was adopted:

- **Dropout:** A dropout rate of 0.1 was applied to the LoRa layers to prevent overfitting.
- **Rank (r):** We experimented with three different values for the rank parameter r: 16, 32, and 64. These relatively low values were chosen because our dataset is small in comparison to the size of the pre-trained models.
- **Alpha (α):** The alpha parameter was set to be twice the rank ($\alpha = 2r$). This scaling factor is used to adjust the importance of the LoRa adapters relative to the original model weights.
- **Adapter Placement:** As demonstrated by the LoRa authors, the placement of adapters playes an important role. While applying LoRa to the query, key, value, and output attention projections is effective, their research indicated that adapting only the **query** and **value** projection matrices offers the best trade-off between performance and parameter efficiency[HSW+22]. Including the key and output matrices resulted in marginal gains at the cost of doubling the number of trainable parameters. Aligning with these findings, we chose to apply the LoRa adapters to the query and value projections, as this approach provides nearly identical performance with significantly fewer parameters.

3.1.6 Results

The training process for the base model required approximately 4 hours for each parameter set, while the large model took 11 hours. The results indicate that an adaptation matrix rank of 32 achieved optimal performance. Lower ranks resulted in insufficient model adaptation, whereas higher ranks led to overfitting. Consequently, a

```
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1,
    bias="none",
)
```

Figure 3.9: Example of LoRA training configuration

matrix rank of 32 yielded the best outcomes for both model variants. All base models experienced early stopping due to a lack of improvement in the loss function.

An analysis of the two best-performing models, based on the Wandb graphics, reveals a significant initial drop in the loss for both. Subsequently, the loss stagnated around 0.19 for the large model and 0.29 for the base model. However, the validation loss, using WER demonstrated a smooth decrease for both models throughout the training process.

Table 3.1: Fine-Tuning Results for Whisper Models

Model	**Rank (r)**	**WER (%)**
whisper-large-v3-turbo	16	8.93
	32	**8.26**
	64	9.5
whisper-base	16	13.49
	32	12.84
	64	15.41

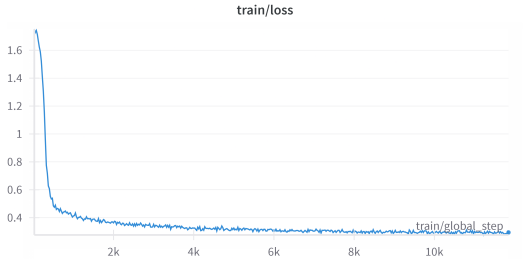
Comparing the results with the original models, we can see that fine-tuning succeeded in the case of the large model, achieving better performance on our validation set.

Table 3.2: Model Word Error Rate (WER) Comparison

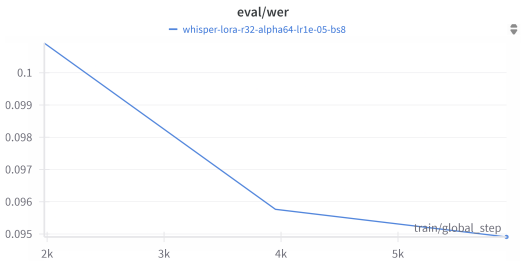
Model Name	**WER (%)**
Whisper Large v3 turbo	8.75
Whisper Large v3 turbo Fine-tuned	**8.26**
Whisper Base	11.37
Whisper Base Fine-tuned	12.84



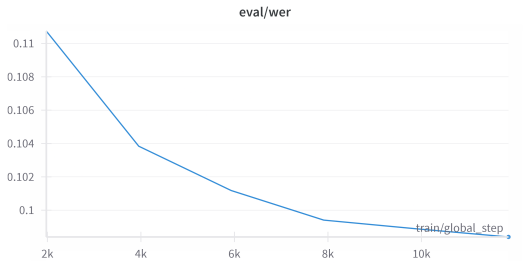
(a) Base Model Train Loss



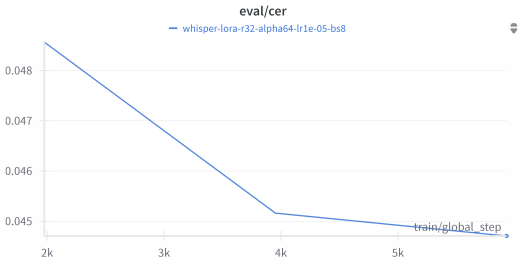
(b) Large Model Train Loss



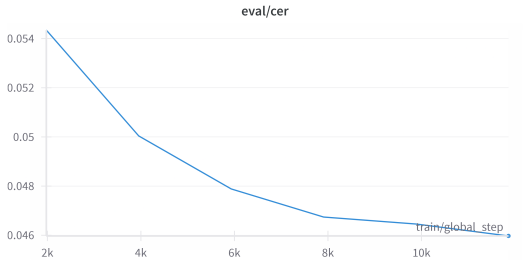
(c) Base Model Validation WER



(d) Large Model Validation WER



(e) Base Model Validation CER



(f) Large Model Validation CER

Figure 3.10: Model Training and Validation Metrics Comparison

3.1.7 Model Deployment for inference

Finally, the last step in our machine learning project lifecycle, to make the system accessible and usable in real-world scenarios, we expose it via a RESTful API and deploy it using a cloud-based infrastructure.

Requirements specification

The transcription service exposes a RESTful interface via a single POST endpoint at `/api/transcription`. Requests to this endpoint must include a body formatted as `multipart/form-data`, containing a single audio file. The file must have a MIME type beginning with `audio/`, and the accepted formats include `.mp3`, `.wav`, and `.webm`. Upon receiving a valid request, the service processes the audio using the preloaded ASR machine learning model.

This API adheres to conventional REST principles and employs standard HTTP status codes for client-server communication. If the transcription is successful, the service responds with an HTTP status code `200 OK` and returns a JSON object containing the transcribed text. In the event of a malformed request—such as missing or unsupported audio content—the service returns an HTTP `400 Bad Request` response. The response content type is always `application/json` and the request must be sent with the header `Content-Type: multipart/form-data`.

Implementation

The transcription service is implemented using `FastAPI`, a Python framework selected for its asynchronous capabilities, high-performance, and support for Python type hints. Its construction was inspired from frameworks like Flask, and Django.

Additional libraries were integrated to support model configuration and audio preprocessing. `PyTorch`[PGM+19] was used to assign the model to a CUDA-enabled GPU and to adjust tensor precision to `float16`, improving inference performance. The `librosa` library[MRL+15] was employed for audio preprocessing tasks, including format conversion and resampling to the expected input specifications. For model loading, adapter integration, and inference configuration, the service relies on the `transformers` library developed by Hugging Face, which provides utilities to load pretrained the base model and incorporate the LoRA adapters into the base model architecture.

The ASR model prepared during the application startup by leveraging FastAPI's `lifespan` mechanism. This allows all model-related resources to be initialized before the service starts accepting requests. The model initialization process loads three main components from the local file system: the base whisper model, the set of LoRA

adapter weights that we fine-tuned, and a processor, which is responsible for feature extranction and tokenization. After the base model and adapter layers are loaded and merged, the model becomes available for inference.

On each request we first verify that the provided data is not missing and is valid. If the audio format is other than mp3, we perform a conversion to mp3. After that we resample the audio file to 16kHz as the wisper models expects. The last step is to segment the audio into 30second chunks, since the whisper model can process only 30 seconds at a time. When the transcription is ready we send it in a JSON response.

For each request, the app first checks that the input data is present and valid. If the audio format is not `.mp3`, it is converted accordingly. The audio is then resampled to 16 kHz, as required by the ASR model. After resampling, the audio is split into 30-second segments to match the model's maximum input duration. Once transcription is complete, the result is returned in a JSON response.

Deployment

For deployment, we used Docker to ensure a reproducible and isolated environment that includes all required Python libraries and dependencies. The application was deployed on Hugging Face Spaces, which provides the necessary hardware support for machine learning inference. Specifically, we used a virtual machine equipped with an NVIDIA T4 GPU and 15 GB of VRAM.

The deployment process began by exporting the project's Python dependencies into a `requirements.txt` file. A corresponding `Dockerfile` was then created to define the steps needed to build the Docker image. We based the image on a PyTorch CUDA-enabled base image to ensure GPU compatibility and proper dependency handling.

The `Dockerfile`, `requirements.txt`, and application source code were uploaded to a remote GitHub repository linked to our Hugging Face Spaces project. Finally, an authentication token was added to the platform's secret manager to restrict access to the endpoint and ensure only authorized users can make requests.

The `Dockerfile`, `requirements.txt`, and application source code were uploaded to a remote GitHub repository linked to the Hugging Face Spaces project. Lastly, to secure access, an authentication token was added to the platform's secret manager, ensuring that only authorized users can interact with the endpoint.

3.2 Web Application

In this chapter, we present the development of *medspeech.io*, a full-stack web application designed to assist medical professionals in recording, transcribing, storing, and

managing patient data efficiently. The primary goal of the application is to provide physicians with a seamless and user-friendly experience in the handling of clinical documentation. *medspeech.io* integrates the ASR API developed in the previous chapter. This Web application aims to improve clinical workflows and minimize the administrative tasks associated with note-taking.

3.2.1 Requirements Specification

This chapter presents the functional and non-functional requirements of the web application for medical professionals.

Functional Requirements

FR-1: Register The system shall provide a registration mechanism for new user accounts, requiring a unique and valid email address.

FR-2: Login The system shall authenticate returning users by sending a single-use, time-sensitive magic link to their registered email address upon a login request.

FR-3: Magic Link Validation The system shall validate the magic link to ensure it is not expired or has not been previously used.

FR-5: Session management The system shall implement session management to keep the user logged in for a defined period.

FR-6: Logout The system shall provide a secure method for users to log out, terminating their session.

FR-6: Patient Management Authenticated users shall be able to create, view, update, and delete patient records. Each patient record must include a name, email address, and phone number, and be persisted in the database.

FR-7: Patient Data Access Control The system shall ensure that doctors can only access and manage patient records that they have personally created.

FR-8: Audio Input for Transcription The system shall allow authenticated users to either (a) initiate an audio recording session directly within the web interface, or (b) upload an existing audio file for transcription. Supported file formats are `.mp3` and `.wav`.

FR-11: Transcription Request After recording or uploading an audio file, the system shall provide an option for the user to initiate a transcription process.

FR-12: Transcription Output Upon successful processing, the system shall return the transcribed text of the submitted audio file to the user.

FR-13: Transcription Management Authenticated users shall be able to create, view, update, and delete their transcriptions. All transcriptions must be persisted in the database.

Non-Functional Requirements

NFR-1 Security: The system shall enforce strict data isolation between tenants (doctors).

NFR-2 Performance: The time to load a doctor's patient list shall not exceed 2 seconds for up to 1,000 patients.

Usability

NFR-2 Usability: The user interface shall be intuitive and responsive, designed for medical professionals who may have varying levels of technical proficiency.

NFR-3 Reliability & Availability: The system shall gracefully handle potential errors, such as failures in the external Speech Recognition API. If the API is unavailable or returns an error, the system must inform the user clearly without crashing.

NFR-4 Scalability: The database schema and queries shall be optimized to handle a large volume of patient records and transcriptions as the user base grows.

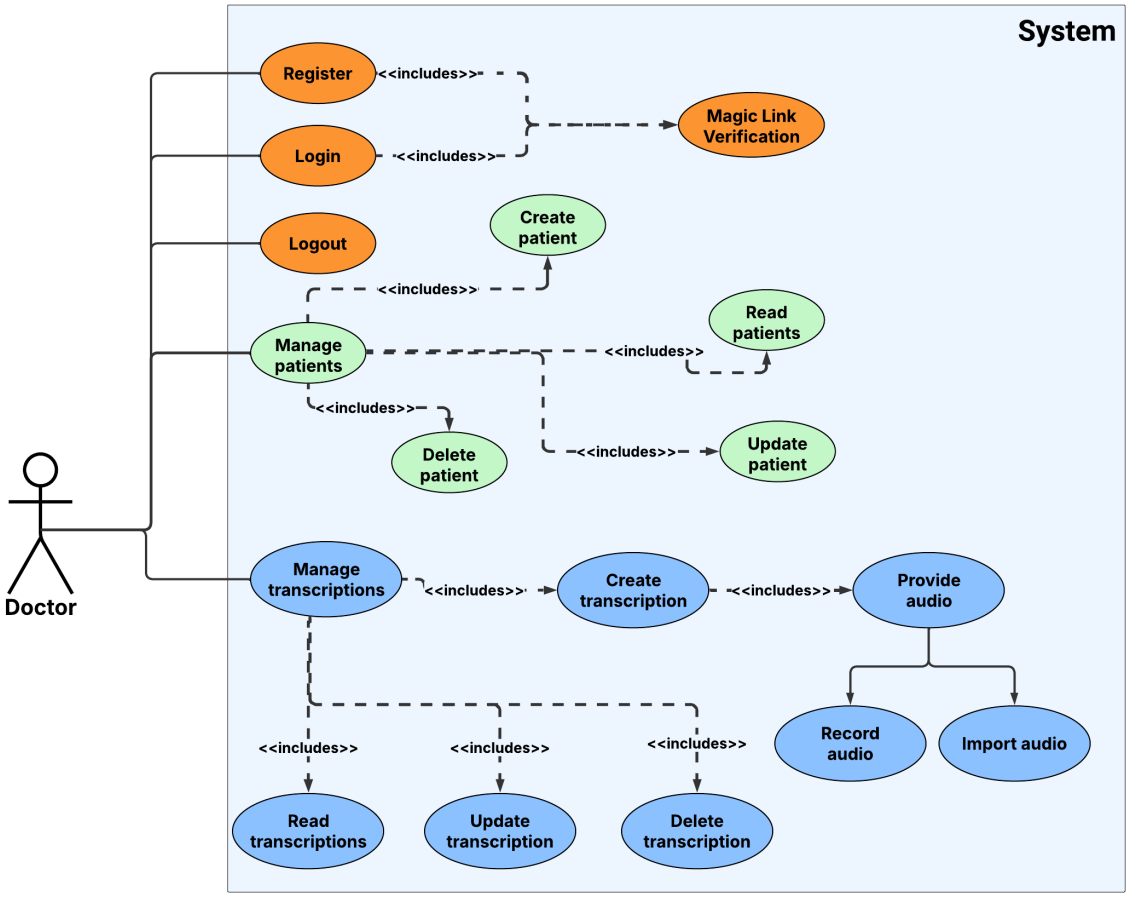


Figure 3.11: Use case diagram

3.2.2 Implementation

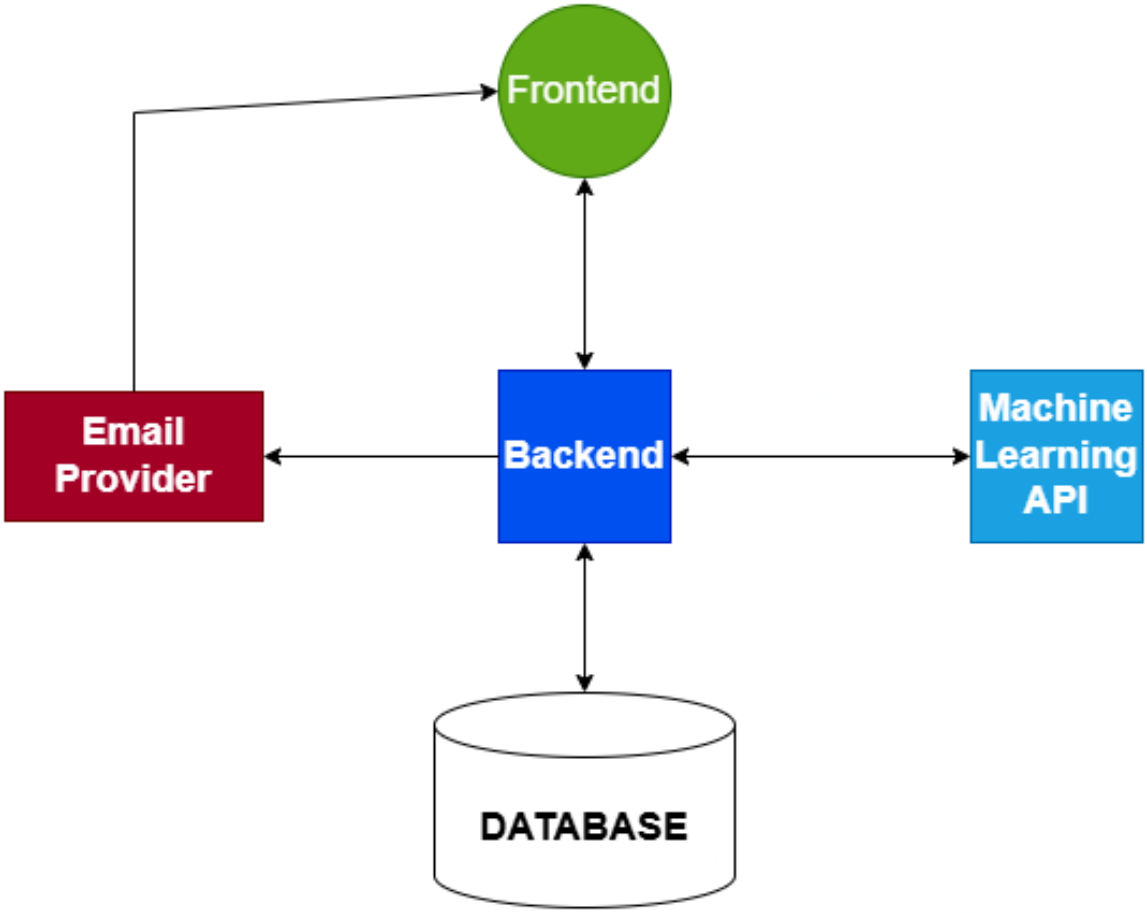


Figure 3.12: Solution Architecture

Backend Module: Serves as the application's central hub and single source of truth, orchestrating all data flows and database communications. The backend handles user authentication via email verification and processes all frontend requests. We implemented this module using NestJS, a Node.js framework that employs MVC architecture patterns borrowed from Spring Framework. Built on TypeScript, NestJS incorporates Angular-inspired Guards to secure exposed endpoints. Database interactions with PostgreSQL are managed through TypeORM, leveraging TypeScript decorators for object-relational mapping.

Frontend Module: Manages user interactions and presents backend data through an intuitive interface. The frontend is constructed with Angular, a TypeScript-based framework, and styled using Tailwind CSS for responsive design implementation.

Machine Learning API: Integrates with the backend to provide the specialized functionality detailed in the previous section, enabling the application's core speech processing capabilities.

Login flow

When a user submits their email, the frontend triggers a REST request to the backend. The backend dispatches an authentication email to the user's inbox containing a verification link with an embedded token as a query parameter. Clicking this link redirects the user to the frontend application, which extracts the token from the URL and forwards it to the backend for validation. Upon successful token verification, the backend returns a JWT token to the client. The frontend employs Angular interceptors to automatically append this JWT to all subsequent requests.

Multi-tenancy

The backend implements a multi-tenant architecture where each request carries a JWT containing the user identifier. All database operations are scoped using this userId, ensuring data isolation between tenants. We established database indexes on the userId field across relevant tables, including patient and conversation entities, to optimize query performance.

Transcription flow

The transcription process begins when a user selects a patient from the interface. Users can either upload an existing audio file or initiate a new recording session. Once the recording concludes or the file uploads, the frontend dispatches a transcription request to the backend. The backend receives the audio data and forwards it to the machine learning API, then waits for the processed transcription response before relaying it back to the client. Upon receiving the transcription, the frontend navigates to a new page displaying the transcription overview.

3.2.3 App Flows



Figure 3.13: Patient and Conversations management Flow

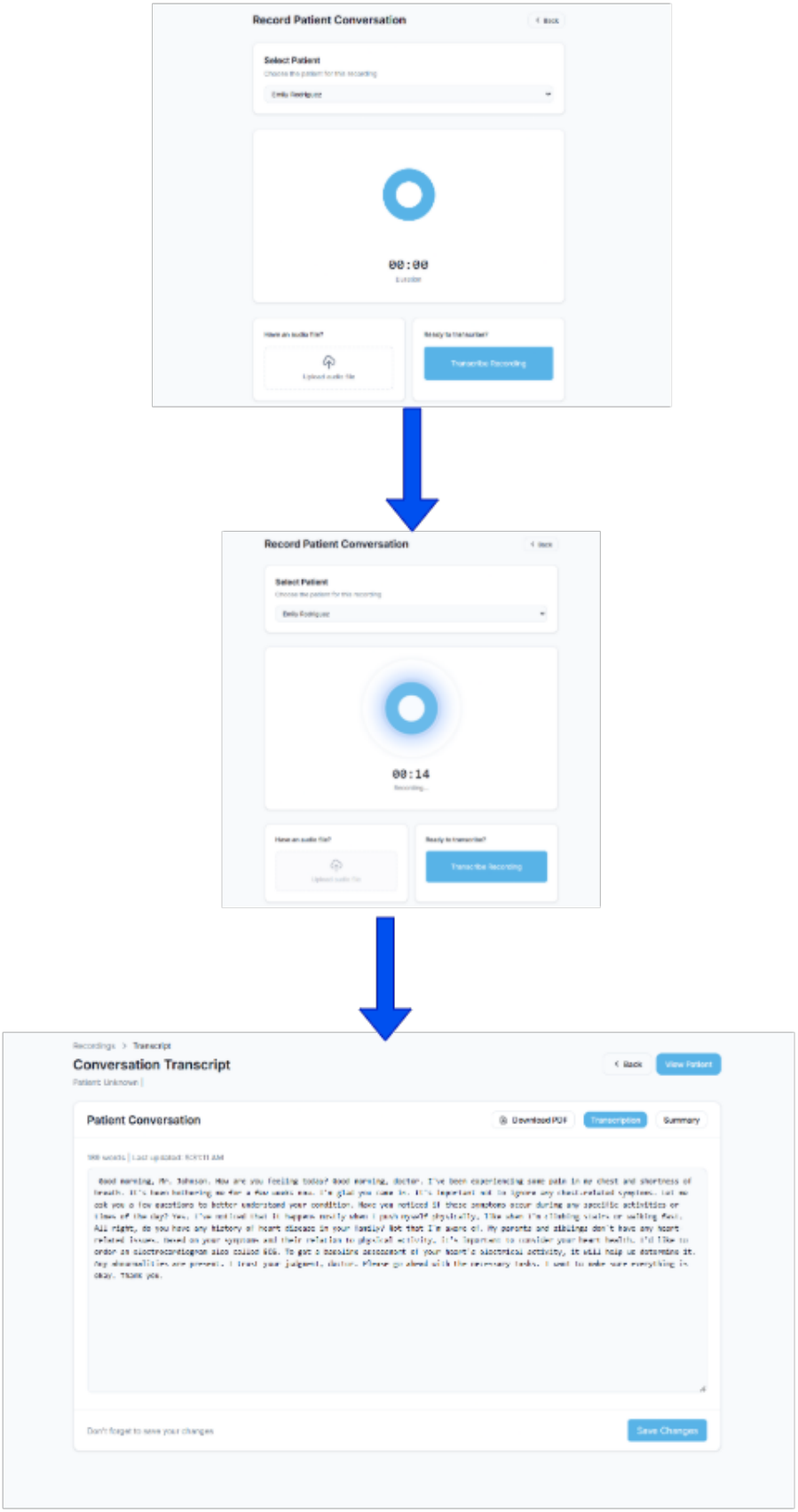


Figure 3.14: The Recording - Transcription Flow

Chapter 4

Conclusions

This thesis titled "ASR for Healthcare: Optimizing Speech Recognition for Medical Conversations" was motivated by the limitations of the current speech recognition systems in handling specialized domains like medicine, where accuracy and domain-specific understanding are critical. An effective medical speech recognition system would enable physicians to automate the transcription of patient consultations, reduce administrative workload, and redirect their focus toward patient care.

To address these limitations, this research aimed to develop a web-based system capable of accurately transcribing audio recordings that include medical terms.

A big challenge that we encountered was the lack of real-world datasets containing paired medical speech and transcriptions, due to strict privacy regulations. All datasets available for this research were synthetic. As a result, the model was not exposed to real-life variations in speech, such as interruptions, background noise, or informal phrasing. Despite this, the Syn-Med dataset was sufficient to prove the concept. Our fine-tuned model achieved a word error rate (WER) of 8.26% on the validation set, compared to 8.75% WER from the original Whisper Large-Turbo base model[RKX+23]. This result is promising, especially considering that we fine-tuned a large, general-purpose, zero-shot model using limited compute. LoRA[HSW+22] was the only viable fine-tuning strategy given our hardware constraints, and it proved to be effective.

To demonstrate a practical use case, we packaged the model into a RESTful API service capable of running inference, and deployed it to the cloud. Finally, we developed *medspeech.io*, a web application that allows physicians to manage patients, access audio conversations, view transcriptions, and track prescriptions.

The solution presented in this thesis still requires significant refinement before it can be deployed in real-world clinical settings, such as during surgical procedures. At present, the model sometimes mis-transcribes specialized terminology when background noise is powerful or when speakers have strong accents. However, this work successfully validates the core idea: starting from a general-purpose base model, it

is possible to enhance transcription accuracy through efficient fine-tuning techniques and a specialized, diverse dataset. Furthermore, the deployment and integration of the improved model into a functional web application demonstrates its practical viability.

4.1 Future work

Some further improvements that would significantly enhance the usability and impact of this project include:

- **Fine-tuning on real-world medical audio**: Further fine-tune the model on de-identified recordings of actual doctor-patient conversations. This would improve robustness by exposing the model to natural speech patterns, diverse accents, interruptions, and real clinical noise.
- **NLP integration**: Incorporate natural language processing tools into *medspeech.io*, such as a large language model capable of automatically summarizing transcripts or extracting key clinical data like medications, dosages, and diagnoses.
- **Speaker diarization**: Integrate an additional machine learning module to perform speaker diarization, enabling the system to distinguish between different speakers. This would allow *medspeech.io* to not only capture what is said, but also identify who is speaking.
- **Clinical usability testing**: Conduct a clinical usability study with healthcare professionals to evaluate how *medspeech.io* impacts their workflow. This feedback would help validate the application in real settings and guide future design and feature improvements.

By considering these enhancements, *medspeech.io* could move from a proof-of-concept to a reliable, medical speech recognition system that supports clinicians in reducing administrative burden and improving the quality and efficiency of clinical documentation.

Bibliography

[BZMA20] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NeurIPS '20, pages 12449–12460. Curran Associates, Inc., 2020.

[Car24] United We Care. United-medsyn: Medical speech dataset for asr, 2024.

[CJLV15] William Chan, Navdeep Jaitly, Quoc V. Le, and Oriol Vinyals. Listen, attend and spell. *arXiv preprint arXiv:1508.01211*, 2015.

[GFGS06] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, Pittsburgh, PA, 2006. ACM.

[HSW+22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[JM25] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript released January 12, 2025.

[MRL+15] Brian McFee, Colin Raffel, Dawen Liang, Daniel P.W. Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. librosa: Audio and music signal analysis in python. In *Proceedings of the 14th Python in Science Conference*, pages 18–25, 2015.

[PGM+19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein,

Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.

[PKMSS22] Alex Papadopoulos Korfiatis, Francesco Moramarco, Radmila Sarac, and Aleksandar Savkov. (in press): Primock57: A dataset of primary care mock consultations. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.

[Rab89] L. R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[RKX+23] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28492–28518. PMLR, 2023.

[SCL+16] Christine Sinsky, Lisa Colligan, Ling Li, Mirela Prgomet, Sandra Reynolds, Lindsey Goeders, and Regan Blocker. Allocation of physician time in ambulatory practice: A time and motion study in 4 specialties. *Annals of Internal Medicine*, 165(11):753–760, 2016.

[SDS+16] Tait D. Shanafelt, Liselotte N. Dyrbye, Christine Sinsky, Osama Hasan, Daniel Satele, Jeff Sloan, and Colin P. West. Relationship between clerical burden and characteristics of the electronic environment with physician burnout and professional satisfaction. *Mayo Clinic Proceedings*, 91(7):836–848, 2016.

[SVL14] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, volume 27, pages 3104–3112. Curran Associates, Inc., 2014.

[Wik25] Wikipedia contributors. Sampling (signal processing). Wikipedia, The Free Encyclopedia, 2025. Accessed: August 17, 2025.

[Wis23] Wisdom ML. Hidden Markov model (HMM) in NLP: Complete implementation in Python, April 2023.